

Atividade – Pipelines no Linux e Middleware CORBA

1. Exemplos de Pipelines no Linux

No Linux, um **pipeline** é uma forma de conectar comandos utilizando o símbolo `|`, chamado de **pipe**. O resultado produzido por um comando é enviado diretamente como entrada para o próximo comando.

Exemplo 1: Listar e filtrar arquivos

```
ls | grep ".txt"
```

Nesse exemplo, o comando `ls` lista os arquivos do diretório atual. O símbolo `|` envia essa lista para o comando `grep`, que filtra e mostra apenas os arquivos que possuem `.txt` no nome.

Esse tipo de pipeline é útil quando existe uma grande quantidade de arquivos e é necessário localizar apenas determinados tipos.

Exemplo 2: Contar arquivos de um diretório

```
ls | wc -l
```

O comando `ls` gera uma lista dos arquivos e diretórios. Essa saída é enviada para o comando `wc -l`, que conta a quantidade de linhas recebidas.

Como normalmente cada item aparece em uma linha, o resultado representa a quantidade de itens listados.

Exemplo 3: Ordenar informações

```
cat nomes.txt | sort
```

Nesse exemplo, o comando `cat` lê o conteúdo do arquivo `nomes.txt`. O resultado é enviado para o comando `sort`, que organiza as linhas em ordem alfabética.

Também é possível criar pipelines com mais de dois comandos, por exemplo:

```
cat nomes.txt | sort | uniq
```

Nesse caso, o conteúdo do arquivo é lido, organizado e, por fim, as linhas repetidas são removidas pelo comando `uniq`.

Conclusão sobre Pipelines

Os pipelines permitem combinar comandos simples para realizar tarefas mais complexas. Cada comando realiza uma função específica e o resultado pode ser utilizado pelo próximo comando.

2. Fluxo de Erro (stderr) em Pipelines no Linux

No Linux, os programas normalmente trabalham com três fluxos principais:

- **stdin (0):** entrada padrão.
- **stdout (1):** saída padrão.
- **stderr (2):** saída de erro.

Quando utilizamos um pipeline com o símbolo `|`, normalmente apenas a saída padrão, ou seja, o **stdout**, é enviada para o próximo comando.

Por exemplo:

```
comando1 | comando2
```

Nesse caso, a saída normal do `comando1` é enviada para o `comando2`. Entretanto, caso aconteça algum erro, a mensagem de erro enviada pelo **stderr** normalmente continua aparecendo no terminal.

Um exemplo é:

```
ls arquivo_inexistente | grep erro
```

Como o arquivo não existe, o comando `ls` gera uma mensagem de erro. Essa mensagem não é enviada automaticamente para o `grep`, pois ela pertence ao fluxo **stderr**.

Para enviar também os erros pelo pipeline, pode-se redirecionar o `stderr` para o `stdout`:

```
comando 2>&1 | outro_comando
```

O trecho `2>&1` significa que o fluxo de erro, identificado pelo número 2, será direcionado para o mesmo local da saída padrão, identificada pelo número 1.

Exemplo:

```
ls arquivo_inexistente 2>&1 | grep "No such file"
```

Agora, tanto a saída normal quanto as mensagens de erro podem ser processadas pelo próximo comando do pipeline.

Portanto, o uso do **stderr** é importante porque permite separar as mensagens normais dos erros. Isso facilita a identificação de problemas e também permite redirecionar ou processar os erros de diferentes formas.

3. Middleware CORBA

CORBA significa **Common Object Request Broker Architecture**, ou Arquitetura Comum de Solicitação de Objetos.

CORBA é uma tecnologia criada para permitir a comunicação entre aplicações distribuídas, mesmo quando elas são desenvolvidas em linguagens de programação ou sistemas operacionais diferentes.

Por exemplo, uma aplicação desenvolvida em Java pode utilizar um serviço criado em C++ por meio de uma arquitetura baseada em CORBA.

Funcionamento

O principal componente do CORBA é o **ORB (Object Request Broker)**.

O ORB funciona como um intermediário entre uma aplicação cliente e um objeto que oferece determinado serviço.

De forma simplificada, o funcionamento ocorre da seguinte maneira:

1. O cliente solicita a execução de uma operação.
2. A solicitação é enviada ao ORB.
3. O ORB localiza o objeto responsável pelo serviço.
4. A solicitação é encaminhada ao objeto.
5. O objeto executa a operação.
6. O resultado é retornado ao cliente.

Dessa forma, o cliente não precisa saber exatamente onde o objeto está localizado ou como ele foi implementado.

IDL

Outro componente importante do CORBA é a **IDL (Interface Definition Language)**.

A IDL é utilizada para definir as interfaces dos objetos distribuídos. Ela permite que aplicações escritas em diferentes linguagens entendam quais operações podem ser utilizadas.

Por exemplo, uma interface pode definir uma operação como:

```
calcular();
```

A implementação dessa operação pode ser feita em uma linguagem, enquanto o cliente pode estar utilizando outra.

Vantagens do CORBA

Entre as principais vantagens do CORBA estão:

- Comunicação entre diferentes linguagens de programação.
- Comunicação entre sistemas distribuídos.
- Independência de plataforma.
- Separação entre cliente e servidor.
- Possibilidade de reutilização de serviços.

Desvantagens

Apesar de suas vantagens, o CORBA também apresenta algumas dificuldades:

- Pode ser complexo de configurar e implementar.
- Possui uma arquitetura considerada pesada para aplicações mais simples.
- Existem tecnologias mais modernas para comunicação entre sistemas, como APIs REST e gRPC.

Conclusão

O CORBA foi uma tecnologia importante para o desenvolvimento de sistemas distribuídos. Seu principal objetivo é permitir a comunicação entre objetos localizados em diferentes sistemas, independentemente da linguagem de programação utilizada.

Apesar de atualmente existirem alternativas mais simples e modernas, o CORBA continua sendo um conceito importante para compreender a evolução dos middlewares e da comunicação em sistemas distribuídos.

Fontes

- GNU Project. GNU Bash Manual.
- GNU Project. Coreutils Manual.
- Oracle. Java Platform – CORBA Documentation.
- Object Management Group (OMG). CORBA Specifications.